



Identificação de Espécies Vegetais com Inteligência Artificial Explicável

Uma aplicação mobile *offline-first* com extensão científica baseada em **Grad-CAM** e nas métricas originais **Attention Leakage (AL)** e **Attention Focus Score (AFS)**.

Autor: Luiz Carlos
Trabalho de Conclusão de Curso — TCC

2026

Resumo

O AgroVision é um aplicativo móvel de identificação de espécies vegetais desenvolvido em React Native, projetado para operar integralmente sem conexão com a internet através de um modelo TensorFlow Lite derivado da arquitetura MobileNetV2. Este trabalho descreve a arquitetura geral do sistema, o pipeline de visão computacional envolvido e, de forma central, a extensão científica adicionada ao projeto para avaliar *qualitativamente e espacialmente* a atenção do modelo, utilizando o método Grad-CAM (Selvaraju et al., 2017) em conjunto com duas métricas originais propostas pelo autor: o **Attention Leakage (AL)**, que quantifica a fração da atenção que recai fora da região de interesse, e o **Attention Focus Score (AFS)**, seu complemento, que mede o grau de alinhamento espacial do modelo com o objeto analisado. Diferentemente de métricas tradicionais de classificação — que reportam apenas o *que* foi predito — AL e AFS descrevem *onde* o modelo olhou, oferecendo uma lente científica complementar, útil para auditar vieses de enquadramento, validar processos de *fine-tuning* e comparar arquiteturas com desempenho classificatório equivalente. A extensão é implementada como um modo científico opt-in, com um serviço FastAPI que expõe o modelo Keras original (necessário porque o TFLite não provê gradientes) e devolve, ao app, o heatmap Grad-CAM, as métricas AL e AFS e uma sobreposição visual para inspeção.

Palavras-chave: Explainable AI; Grad-CAM; Visão Computacional; MobileNetV2; Identificação de Espécies; Attention Leakage; React Native; TensorFlow Lite.

Sumário

1	1. Introdução e motivação
2	2. Arquitetura geral do sistema
3	3. Stack tecnológica
4	4. Estrutura do projeto
5	5. Modelo de visão computacional
6	6. Pipeline de treinamento
7	7. Fundamentação — Explainable AI e Grad-CAM
8	8. ★ Attention Leakage (AL) e Attention Focus Score (AFS) — contribuição original
9	9. Estratégias de máscara para o cálculo de AL
10	10. Arquitetura do modo científico (XAI)
11	11. Implementação do Grad-CAM sobre o modelo Keras
12	12. Persistência e exportação de dados
13	13. Diagrama de casos de uso
14	14. Fluxograma do sistema
15	15. Discussão e trabalhos futuros
16	16. Referências

1. Introdução e motivação

A agricultura moderna, e em particular a chamada *agricultura de precisão*, tem absorvido ferramentas de visão computacional em ritmo crescente para tarefas que vão da identificação taxonômica de espécies cultivadas à detecção de pragas e doenças foliares. Em um cenário de uso real — o agricultor em campo, o estudante em aula prática, o botânico em expedição — a conectividade com a internet é intermitente, o que restringe soluções puramente baseadas em nuvem. O AgroVision nasce dessa constatação: um aplicativo móvel desenvolvido em React Native com Expo, cuja identificação de espécies opera *integralmente offline* através de um modelo TensorFlow Lite embarcado.

Porém, a acurácia de um modelo classificatório, por mais elevada que seja, não conta toda a história. Um modelo pode atingir 95% de precisão em um conjunto de validação e *ainda assim* estar olhando para as pistas erradas — um viés de fundo, uma marca d'água, um padrão de iluminação específico do dataset de origem. A questão científica central que este trabalho aborda é, portanto, complementar à acurácia:

“Dado que o modelo acertou, onde, espacialmente, ele olhou para tomar essa decisão? E o quanto dessa atenção está alinhada com o objeto de interesse?”

Para responder essa pergunta, o AgroVision foi estendido com um **Modo Científico** — um canal opt-in, online, que executa o algoritmo Grad-CAM sobre o modelo Keras original e reporta, além da predição usual, duas métricas espaciais propostas pelo autor: o **Attention Leakage (AL)** e o **Attention Focus Score (AFS)**. AL e AFS descrevem, em termos geométricos, a fração da atenção do modelo que está *fora* e *dentro* da região onde se esperaria que o objeto estivesse. Essa lente complementar permite ao pesquisador auditar comportamento espacial, comparar diferentes estratégias de treinamento e quantificar o efeito de enquadramento nas fotos fornecidas ao modelo.

2. Arquitetura geral do sistema

O sistema é composto por três blocos independentes, orquestrados de modo que o caminho de inferência comum preserve sua característica *offline-first*, enquanto a camada de análise científica, por exigir gradientes do modelo, é fornecida separadamente por um serviço Python local:

Arquitetura Geral do Sistema AgroVision

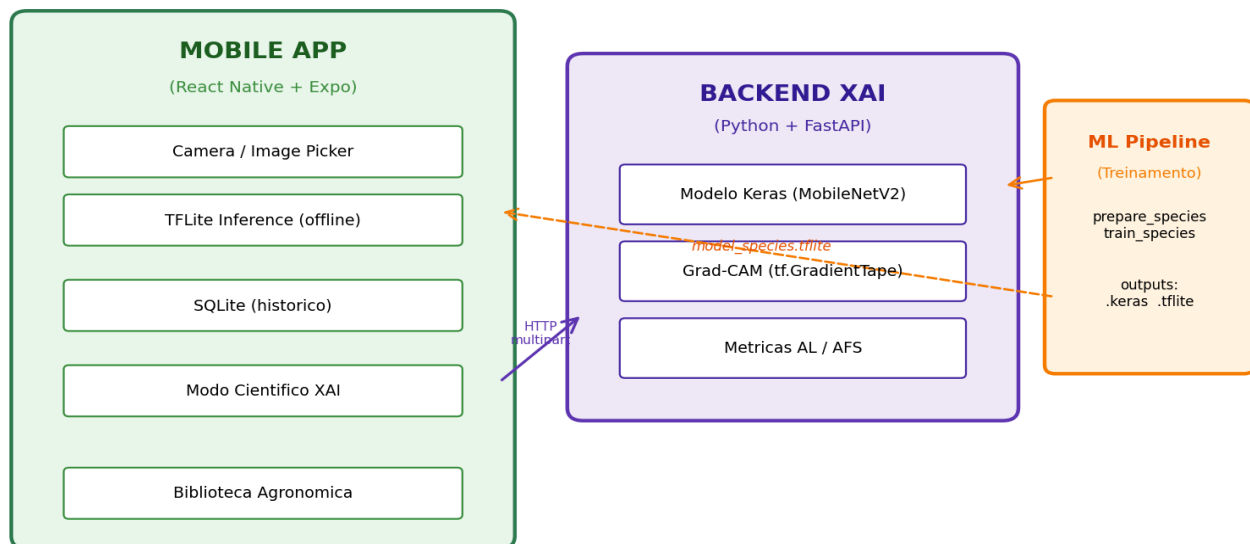


Figura 1. Arquitetura macro do AgroVision. A linha tracejada indica o fluxo de artefatos do pipeline de treinamento: o modelo Keras é consumido pelo backend XAI e, em paralelo, convertido em TFLite para embarque no app.

Essa decomposição reflete uma decisão arquitetural deliberada: **o TFLite, por construção, é um runtime otimizado para inferência e não expõe gradientes**, o que torna impossível aplicar Grad-CAM diretamente sobre ele. A alternativa escolhida foi preservar o modelo Keras original como artefato de primeira classe (salvo no disco durante o treinamento) e disponibilizá-lo ao app através de um endpoint HTTP. O resultado é uma separação limpa: o usuário comum nunca depende da rede para fazer uma identificação, enquanto o pesquisador, no modo científico, conecta-se ao backend para obter o heatmap e as métricas AL/AFS.

3. Stack tecnológica

A stack foi escolhida priorizando (i) a capacidade de rodar inferência pesada em dispositivos modestos, (ii) a facilidade de desenvolvimento e iteração em um trabalho acadêmico individual e (iii) compatibilidade entre os binários gerados pelo treinamento TensorFlow e os runtimes mobile.

Camada	Tecnologia	Versão	Papel
Mobile UI	React Native	0.81.5	Runtime nativo iOS/Android
	Expo	~54	Toolchain + OTA + APIs nativas
	TypeScript	~5.3	Tipagem estática
	React Navigation	~6	Roteamento (tabs + stack)
Mobile ML	react-native-tflite	0.0.2	Inferência offline embarcada
Mobile Storage	expo-sqlite	~15	Histórico + testes XAI
	expo-file-system	~17	Exportação CSV
	expo-sharing	~12	Compartilhamento de arquivos
Mobile Captura	expo-camera	~17	Captura de foto
	expo-image-picker	~15	Seleção múltipla (modo XAI)
Backend XAI	Python	3.10 / 3.11	Runtime
	FastAPI	0.110	HTTP framework assíncrono
	Uvicorn	0.29	ASGI server
ML Training	TensorFlow	2.15.0	Treinamento + Keras + TFLite
	OpenCV	4.8	CLAHE, HSV mask, overlay
	NumPy / Pillow	1.26 / 10	Tensores e I/O de imagem
Datasets	HuggingFace datasets	2.18	Streaming PlantNet 300K + PlantVillage

Tabela 1. Principais dependências do projeto e seus papéis.

4. Estrutura do projeto

O repositório se organiza em dois diretórios paralelos, refletindo a separação arquitetural descrita na seção anterior. Itens marcados como *NOVO* foram introduzidos pela extensão XAI descrita neste documento.

```
Luiz/
■ ■ ■ ■ Agrovision/ # Aplicativo mobile (RN + Expo)
■ ■ ■ ■ src/
■ ■ ■ ■ components/
■ ■ ■ ■ ■ ui/ # Button, Card, LoadingSpinner, ErrorBanner
■ ■ ■ ■ ■ common/ # SafeAreaView, Header
■ ■ ■ ■ ■ xai/ # HeatmapOverlay, MetricsPanel ← NOVO
■ ■ ■ ■ screens/
■ ■ ■ ■ ■ HomeScreen.tsx
■ ■ ■ ■ ■ CameraScreen.tsx
■ ■ ■ ■ ■ ProcessingScreen.tsx
■ ■ ■ ■ ■ ResultScreen.tsx
■ ■ ■ ■ ■ InstitutionalScreen.tsx
■ ■ ■ ■ ■ TestModeScreen.tsx ← NOVO
■ ■ ■ ■ services/
■ ■ ■ ■ ■ ml/tfliteService.ts # inferência offline
■ ■ ■ ■ ■ ml/xaiService.ts # cliente do backend XAI ← NOVO
■ ■ ■ ■ ■ storage/database.ts # SQLite (analyses + xai_tests)
■ ■ ■ ■ ■ api/client.ts
■ ■ ■ ■ models/
■ ■ ■ ■ ■ xai.ts # tipos do modo científico ← NOVO
■ ■ ■ ■ ■ utils/csvExport.ts ← NOVO
■ ■ ■ ■ navigation/
■ ■ ■ ■ assets/models/
■ ■ ■ ■ ■ model_species.tflite # consumido pelo app
■ ■ ■ ■ ■ labels_species.json
■ ■ ■ ■ docs/generate_report.py # este gerador
■
■ ■ ■ ■ agrovision-ml-service/ # Pipeline de ML + backend XAI
■ ■ ■ ■ ■ prepare_species.py # baixa datasets HuggingFace
■ ■ ■ ■ ■ train_species.py # treina e exporta .keras + .tflite
■ ■ ■ ■ ■ model_species.keras # consumido pelo backend ← NOVO
■ ■ ■ ■ ■ labels_species.json
■ ■ ■ ■ requirements.txt
■ ■ ■ ■ api/
■ ■ ■ ■ ■ server.py # FastAPI app ← NOVO
■ ■ ■ ■ xai/
■ ■ ■ ■ ■ model_loader.py
■ ■ ■ ■ ■ gradcam.py # tf.GradientTape
■ ■ ■ ■ ■ mask.py # fixed / green / manual
■ ■ ■ ■ ■ metrics.py # AL e AFS
■ ■ ■ ■ overlay.py # JET overlay
```

5. Modelo de visão computacional

O classificador de espécies é construído sobre a arquitetura **MobileNetV2** (Sandler et al., 2018), escolhida por três razões convergentes: (i) seu compromisso favorável entre acurácia e custo computacional, viabilizando execução no dispositivo móvel sem GPU; (ii) a existência de pesos pré-treinados em ImageNet, permitindo *transfer learning* com um conjunto de dados botânico de escala moderada; e (iii) seu bloco final contendo convoluções 2D seguidas de *global average pooling*, topologia que se presta naturalmente ao Grad-CAM.

5.1. Topologia

A cabeça classificatória adicionada sobre o *backbone* é minimalista:

```
Input (224x224x3) → preprocess_input → MobileNetV2 (frozen / unfrozen) →  
GlobalAveragePooling2D → Dropout(0.5) → Dense(N_classes, softmax)
```

5.2. Pré-processamento (CLAHE)

Durante a preparação do dataset, cada imagem é submetida ao filtro **CLAHE** (Contrast Limited Adaptive Histogram Equalization, Zuiderveld, 1994), que realça estruturas foliares sem amplificar ruído, e em seguida recebe *padding* proporcional para alcançar a geometria 224×224 sem deformação. Esse tratamento reduz a variância causada por diferenças de iluminação entre as fontes de imagens (PlantNet 300K e PlantVillage).

5.3. Data augmentation

Transformações estocásticas são aplicadas *somente* durante o treino, em um `tf.data.Dataset` à parte — nunca embutidas no modelo. Isso garante um grafo limpo e determinístico no momento da conversão para TFLite. As transformações incluem *flip horizontal/vertical*, rotação aleatória ($\pm 25\%$), zoom, translação, variação de brilho e contraste.

6. Pipeline de treinamento

Pipeline de Treinamento — do dataset ao modelo dual (Keras + TFLite)

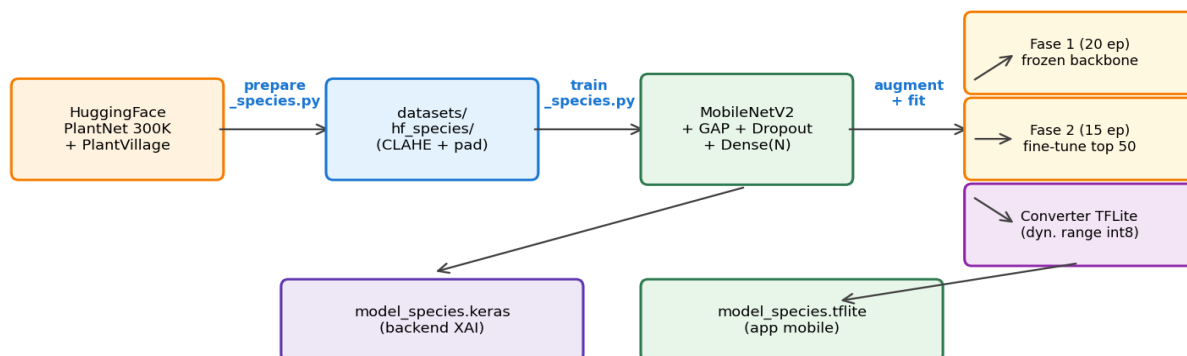


Figura 2. Pipeline de treinamento — do download dos datasets HuggingFace até os dois artefatos finais: *model_species.keras* (para o backend XAI) e *model_species.tflite* (para o app).

6.1. Treinamento em duas fases

O script `train_species.py` executa o treinamento em duas fases sequenciais, prática consolidada em *transfer learning* com redes pré-treinadas:

- **Fase 1 — *feature extraction*:** o *backbone* MobileNetV2 é congelado e apenas o cabeçalho denso é treinado (20 épocas, *learning rate* = 10^{-3}). Essa fase é barata computacionalmente e adapta rapidamente o espaço latente pré-existente ao domínio botânico.
- **Fase 2 — *fine-tuning*:** as 50 últimas camadas do *backbone* são descongeladas e o treino continua com *learning rate* reduzido (10^{-4}) por mais 15 épocas, permitindo que as convoluções tardias se especializem em padrões foliares específicos do dataset.

6.2. Conversão para TFLite

Concluído o treinamento, o modelo Keras é convertido para TFLite com `Optimize.DEFAULT`, o que habilita *dynamic range quantization* — pesos são armazenados em int8 com fator de escala, enquanto as ativações permanecem em float32. Essa combinação gera um artefato compacto (~10 MB para ~150 classes) com perda de acurácia tipicamente inferior a 1 ponto percentual, adequada ao runtime móvel.

Uma contribuição recente a este pipeline foi a inclusão explícita da instrução `model.save('model_species.keras')` imediatamente antes da conversão — passo necessário para viabilizar o backend XAI, que exige o modelo Keras completo para computar gradientes.

7. Fundamentação — Explainable AI e Grad-CAM

Explainable AI (XAI) é a subárea da aprendizagem de máquina que busca tornar as decisões de modelos — especialmente redes neurais profundas — inteligíveis a seres humanos. Entre os métodos mais influentes para explicar redes convolucionais de classificação de imagens está o **Grad-CAM** (Gradient-weighted Class Activation Mapping), introduzido por Selvaraju et al. (2017). Grad-CAM produz, para uma imagem de entrada e uma classe-alvo c , um mapa 2D do tamanho do último bloco convolucional que indica quais regiões da imagem mais contribuíram para a decisão do modelo.

7.1. Formulação matemática

Seja A^k o k -ésimo mapa de ativação (*feature map*) da última camada convolucional de uma rede, e y^c o *score* (ou probabilidade pré-softmax) da classe c . Define-se o peso de importância do canal k como o valor médio das derivadas parciais de y^c em relação a cada elemento de A^k :

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$

onde Z é o número total de posições ($H \cdot W$) em A^k . O mapa de ativação de classe Grad-CAM é então obtido pela combinação linear dos mapas de ativação, ponderados pelos coeficientes calculados, seguida de um *rectifier*:

$$L_{\text{GradCAM}}^c(x, y) = \text{ReLU}\left(\sum_k \alpha_k^c A^k(x, y)\right)$$

A operação ReLU descarta contribuições negativas — regiões que *reduzem* a pontuação da classe-alvo — mantendo apenas o sinal positivo, i.e., as regiões que *suportam* a decisão. Posteriormente, o heatmap é normalizado para o intervalo $[0, 1]$ e redimensionado para a resolução da imagem original, o que permite sobrepô-lo visualmente à foto para inspeção.

7.2. Por que Grad-CAM é adequado ao AgroVision

MobileNetV2 termina com uma convolução *pointwise* (tipicamente chamada Conv_1) seguida de *global average pooling* e da camada densa classificatória. Essa topologia é *exatamente* o cenário em que Grad-CAM foi formulado — a GAP garante que os gradientes propagados até Conv_1 sejam representativos das contribuições espaciais, e a última camada convolucional carrega a granularidade visual suficiente (7×7 para entradas 224×224) para produzir mapas de atenção interpretáveis.

Pipeline do Grad-CAM + metricas XAI (AL, AFS)

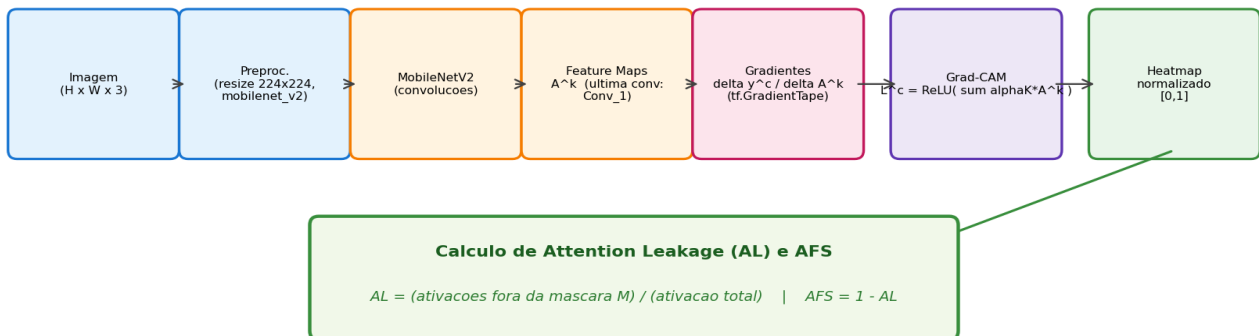


Figura 3. Pipeline do Grad-CAM integrado ao AgroVision. A saída (heatmap normalizado) alimenta o cálculo das métricas AL e AFS descritas na seção 8.

8. Attention Leakage (AL) e Attention Focus Score (AFS)

Esta é a contribuição original deste trabalho. Grad-CAM responde *onde* o modelo olhou, mas por si só não quantifica o quanto essa atenção está *correta* do ponto de vista espacial. Propõe-se, portanto, um par de métricas complementares que **reduzem o heatmap Grad-CAM a dois escalares interpretáveis**, permitindo comparações objetivas entre imagens, modelos ou estratégias de treinamento.

8.1. Motivação

Métricas tradicionais de classificação — acurácia, precisão, F1, matriz de confusão — operam sobre a saída escalar do modelo e descrevem *o que* foi predito. Elas são cegas ao *onde*: dois modelos com a mesma acurácia podem ter comportamentos espaciais radicalmente diferentes. Um poderia olhar majoritariamente para a planta; outro poderia acertar por correlação com o fundo, um padrão de iluminação ou uma assinatura do dataset de origem. No contexto agrícola essa distinção é operacionalmente relevante — um modelo que generaliza espacialmente bem tende a ser mais robusto a enquadramentos de campo, mudanças de iluminação e diferentes dispositivos de captura.

8.2. Definições formais

Seja $H(x,y) \in [0, 1]$ o heatmap Grad-CAM normalizado, avaliado em cada pixel (x,y) da imagem original de dimensões $(W \times H)$. Seja $M(x,y) \in \{0, 1\}$ uma máscara binária que delimita a região de interesse (RoI) — tipicamente, o *bounding box* da planta alvo da análise.

Introduz-se um *threshold* $\tau \in [0, 1]$ que elimina ativações marginais, consideradas ruído, produzindo o heatmap truncado $H'(x,y)$:

$$H'(x,y) = H(x,y) \text{ se } H(x,y) \geq \tau, \text{ senão } 0$$

Define-se então o **Attention Leakage** como a fração da atenção pós-threshold que recai *fora* da máscara:

$$AL = \frac{\sum_{(x,y) \text{ fora de } M} H'(x,y)}{\sum_{(x,y)} H'(x,y)}$$

e, simetricamente, o **Attention Focus Score** como seu complemento:

$$AFS = 1 - AL$$

8.3. Propriedades matemáticas

- **Complementaridade:** por construção, $AL + AFS = 1$ em qualquer imagem. AFS é, portanto, redundante do ponto de vista informacional, mas sua leitura é mais natural (quanto maior, melhor).
- **Domínio:** $AL, AFS \in [0, 1]$. Valores intermediários admitem leitura percentual direta.
- **Invariância a escala linear:** como AL é uma razão, multiplicar todo o heatmap por uma constante positiva não altera o valor — o que é desejável dado que Grad-CAM já normaliza para $[0, 1]$ mas diferenças de magnitude entre amostras deveriam ser irrelevantes para comparações.
- **Caso degenerado:** se $H'(x,y) = 0$ em toda a imagem — situação possível quando o modelo não ativa acima do threshold — define-se, por convenção, $AL = 0$ e $AFS = 1$, evitando divisão por zero.
- **Sensibilidade a τ :** thresholds baixos ($\tau \rightarrow 0$) consideram quase toda a ativação e aumentam o ruído em regiões irrelevantes; thresholds altos ($\tau \rightarrow 1$) concentram a avaliação nos picos, tornando a métrica mais robusta mas potencialmente subsensível. $\tau = 0.5$ é adotado como *default* por produzir bom equilíbrio empiricamente.

8.4. Interpretação qualitativa

Um AFS próximo de 1 indica que o modelo concentrou sua atenção dentro da região esperada — seja porque o enquadramento da foto era adequado, seja porque o modelo aprendeu a isolar a planta. Valores intermediários ($0.4 \leq AFS < 0.7$) sugerem que parte relevante da atenção vazou para o fundo, o que pode ser aceitável em cenários com plantas pequenas em cenas amplas, mas alerta para possível viés ao ser cruzado com erros de predição. Valores baixos ($AFS < 0.4$) sinalizam que o modelo tomou a decisão por pistas contextuais, não pelo objeto — um padrão especialmente perigoso em datasets com viés sistemático.

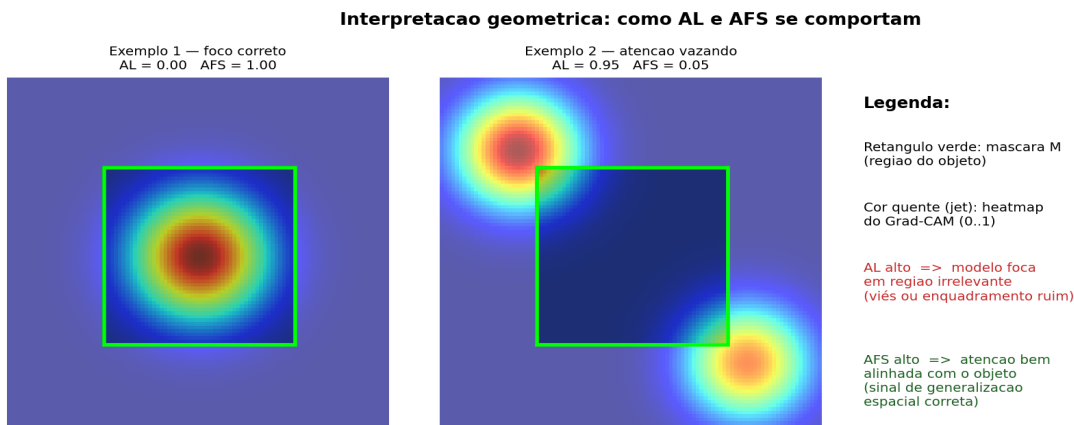


Figura 4. Interpretação geométrica das métricas. À esquerda, um heatmap bem concentrado na região objeto (quadrado verde) resulta em AFS elevado. Ao centro, dois focos de ativação fora da máscara produzem AL alto.

8.5. Casos de uso científicos

- **Auditoria de viés:** comparar AFS médio em subconjuntos do dataset onde a planta ocupa diferentes porções do quadro. Se o AFS cai acentuadamente em imagens com planta pequena, há indício de dependência do modelo em pistas de fundo.

-
- **Validação de *fine-tuning*:** espera-se que, ao refinar o modelo com dados mais representativos da condição de campo, o AFS médio aumente — indicando que o modelo passou a localizar melhor o objeto, mesmo que a acurácia global já estivesse saturada.
 - **Comparação entre arquiteturas:** duas redes com acurácia equivalente podem ter AFS distintos. A de maior AFS tende a ser preferível operacionalmente, por sua robustez espacial.
 - **Enquadramento no app:** comparar AFS em imagens tiradas pelo usuário final (bem e mal enquadradas) permite decidir se vale introduzir um guia visual de captura, reduzindo erros em produção.

9. Estratégias de máscara para o cálculo de AL

A máscara M define o que se entende por “região do objeto” e é, portanto, determinante para a interpretação de AL/AFS. Três estratégias são oferecidas pelo backend e selecionáveis pela interface:

9.1. Bbox central fixo (*fixed*)

Um retângulo centralizado cobrindo 80% da imagem em cada dimensão. É a estratégia mais simples e serve como *baseline*. Em imagens razoavelmente enquadradas — situação típica quando o próprio usuário captura a planta no centro — a cobertura de 80% tende a envolver o objeto sem incluir muito fundo. Essa estratégia é rápida e determinística.

9.2. Segmentação por verde (*green*)

Imagem é convertida para HSV e limiarizada em uma faixa ampla de verde (matiz 25–95), cobrindo folhagem de tons variados. A máscara é limpa por operações morfológicas (abertura + fechamento com *kernel* elíptico 5×5) e o maior componente conexo é extraído como bounding box do objeto. Se nenhum componente atinge 2% da área da imagem, o sistema recua graciosamente para a estratégia *fixed*, reportando o fallback na resposta HTTP. Essa heurística é adequada para fotos com planta razoavelmente isolada e é muito mais precisa do que a estratégia fixa para compor a RoI.

9.3. Manual

O usuário fornece um bbox no formato x,y,w,h . Essa estratégia é útil quando se deseja avaliar AL sobre uma sub-região específica — por exemplo, uma única folha dentro de uma planta maior.

10. Arquitetura do modo científico (XAI)

O modo científico é acessado a partir de um cartão dedicado na tela principal do app. Ao abrir a tela `TestModeScreen`, o app faz uma verificação de saúde contra o endpoint `GET /api/health` — se o backend não responde, exibe instruções de setup. Em seguida, o usuário pode:

- Selecionar uma ou várias imagens da galeria (multi-seleção, até 20 por lote).
- Escolher a estratégia de máscara (*fixed* ou *green*) e o *threshold* τ (valores pré-definidos: 0.3, 0.5, 0.7).
- Executar o processamento — cada imagem é enviada via HTTP *multipart* para `POST /api/xai/gradcam`. O backend retorna predição, confiança, AL, AFS, a bbox efetivamente usada e a imagem com o overlay Grad-CAM codificada em *base64*.
- Rotular manualmente o enquadramento de cada imagem como *bem_enquadrada*, *mal_enquadrada* ou *nao_classificada*. Esse rótulo é opcional e serve para análise estatística posterior.
- Exportar o histórico completo em CSV — o arquivo é gerado localmente no cache do app e aberto no *share sheet* do sistema operacional.

Todos os resultados são persistidos em uma tabela SQLite independente (`xai_tests`), preservando o histórico entre sessões sem interferir no histórico do fluxo normal de identificação.

11. Implementação do Grad-CAM sobre o modelo Keras

O núcleo do Grad-CAM reside em duas operações: acessar os *feature maps* da última camada convolucional e obter os gradientes da pontuação da classe-alvo em relação a esses mapas. Ambas exigem o modelo Keras íntegro, o que só é possível graças ao artefato `model_species.keras` salvo durante o treinamento. O TensorFlow expõe essa capacidade através do mecanismo `tf.GradientTape`:

```
with tf.GradientTape() as tape:
    feature_maps, predictions = grad_model(input_tensor, training=False)
    class_index = int(tf.argmax(predictions[0]).numpy())
    class_score = predictions[:, class_index]

    grads = tape.gradient(class_score, feature_maps)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2)) # alpha_k^c

    cam = tf.reduce_sum(feature_maps[0] * pooled_grads, axis=-1)
    cam = tf.nn.relu(cam) # mantém so contribuicoes positivas
    cam = cam / (tf.reduce_max(cam) + 1e-8) # normaliza para [0,1]
    cam = tf.image.resize(cam[..., None], (H, W)).numpy().squeeze(-1)
```

Uma complicação prática surge do fato de o *backbone* MobileNetV2 estar embutido como sub-modelo dentro do modelo de classificação. O método `_build_grad_model` localiza recursivamente a camada convolucional alvo (tipicamente `Conv_1`), reconstrói o grafo de maneira a expor tanto os *feature maps* quanto a saída final do modelo em um único passe *forward*, e devolve esse modelo auxiliar ao algoritmo principal.

Para o cálculo de AL/AFS, a implementação é deliberadamente direta:

```
active = np.where(heatmap >= threshold, heatmap, 0.0)
inside = float((active * mask).sum())
outside = float((active * (1.0 - mask)).sum())
total = inside + outside

al = outside / total if total > 0 else 0.0
afs = 1.0 - al
```

O *threshold* é aplicado por `np.where`, zerando elementos abaixo de τ e preservando o valor original dos demais. A multiplicação elemento a elemento com a máscara binária separa as contribuições dentro e fora da RoI, e a razão final produz AL. O caso degenerado (ativação total zero) é tratado explicitamente.

12. Persistência e exportação de dados

Cada execução do modo científico é armazenada na tabela `xai_tests` do SQLite local do aplicativo. O esquema preserva os metadados necessários para análise posterior sem depender do backend — o *overlay* é guardado em base64 para que o usuário possa revisitar resultados anteriores sem necessidade de re-executar o Grad-CAM.

```
CREATE TABLE IF NOT EXISTS xai_tests (  
  id TEXT PRIMARY KEY,  
  imageUrl TEXT NOT NULL,  
  prediction TEXT NOT NULL,  
  confidence REAL NOT NULL,  
  al REAL NOT NULL,  
  afs REAL NOT NULL,  
  threshold REAL NOT NULL,  
  bboxJson TEXT NOT NULL,  
  bboxStrategy TEXT NOT NULL,  
  overlayB64 TEXT NOT NULL,  
  label TEXT NOT NULL,  
  createdAt TEXT NOT NULL  
);
```

A exportação em CSV, implementada em `utils/csvExport.ts`, gera um arquivo com uma linha por teste contendo as colunas: *id*, *createdAt*, *prediction*, *confidence*, *al*, *afs*, *threshold*, *bboxStrategy*, *bboxX*, *bboxY*, *bboxW*, *bboxH*, *label*, *imageUrl*. O arquivo é escrito no diretório de cache do app e compartilhado através do *share sheet* do sistema operacional, permitindo integração imediata com ferramentas analíticas externas (planilhas, Pandas, R).

13. Diagrama de casos de uso

O diagrama de casos de uso explicita a divisão de responsabilidades entre os dois perfis de usuário atendidos pelo sistema. O **Usuário** (botânico, estudante, agricultor) interage com o fluxo tradicional de identificação, inteiramente offline. O **Pesquisador** — tipicamente o autor ou leitores deste trabalho em contexto de análise científica — ativa o modo XAI e interage com casos de uso específicos de avaliação de atenção e exportação de dados.

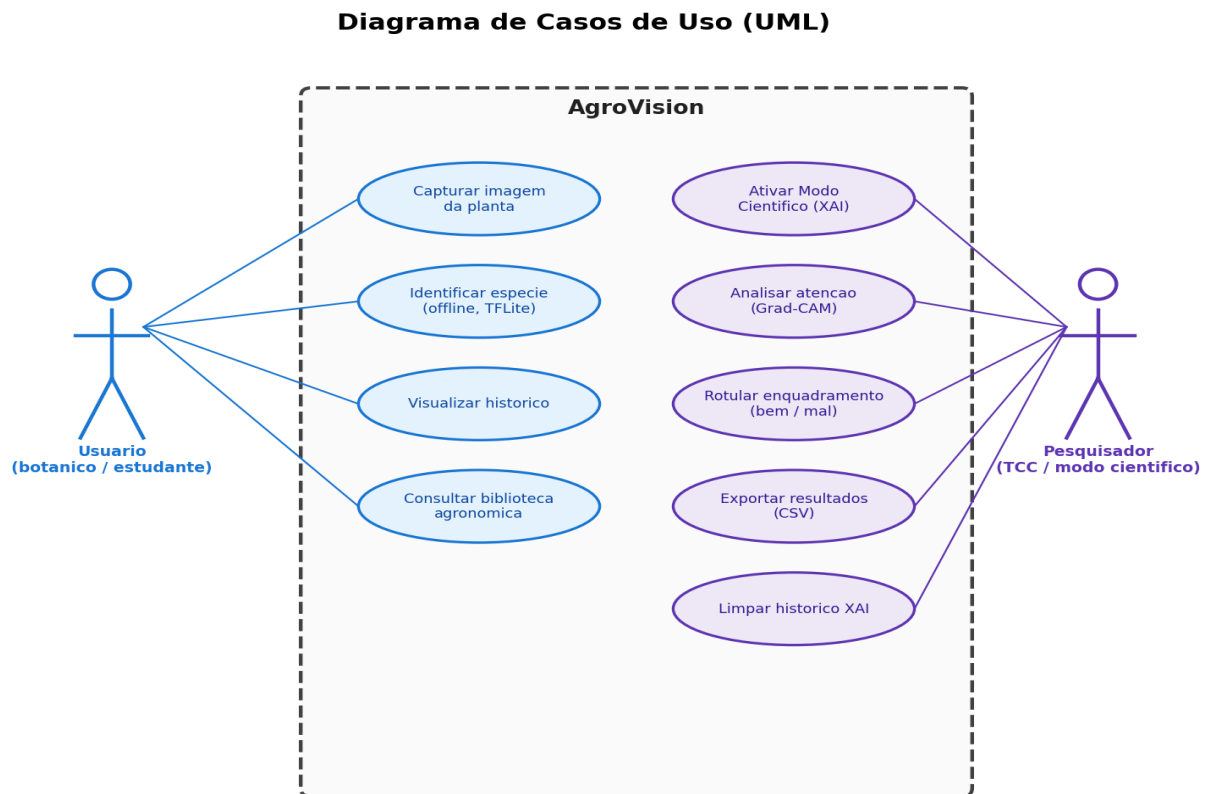


Figura 5. Casos de uso. As elipses em azul representam funcionalidades consumidas pelo usuário comum; as em roxo, aquelas acessíveis apenas a partir do modo científico.

14. Fluxograma do sistema

O fluxograma a seguir descreve, em nível macro, os dois caminhos concorrentes do sistema a partir da tela inicial. O caminho à esquerda é o fluxo canônico de identificação, concluído integralmente no dispositivo; o caminho à direita, condicional à disponibilidade do backend XAI, conduz ao modo científico com persistência e exportação de resultados.

Fluxograma do Sistema — Fluxo Normal e Modo Científico (XAI)

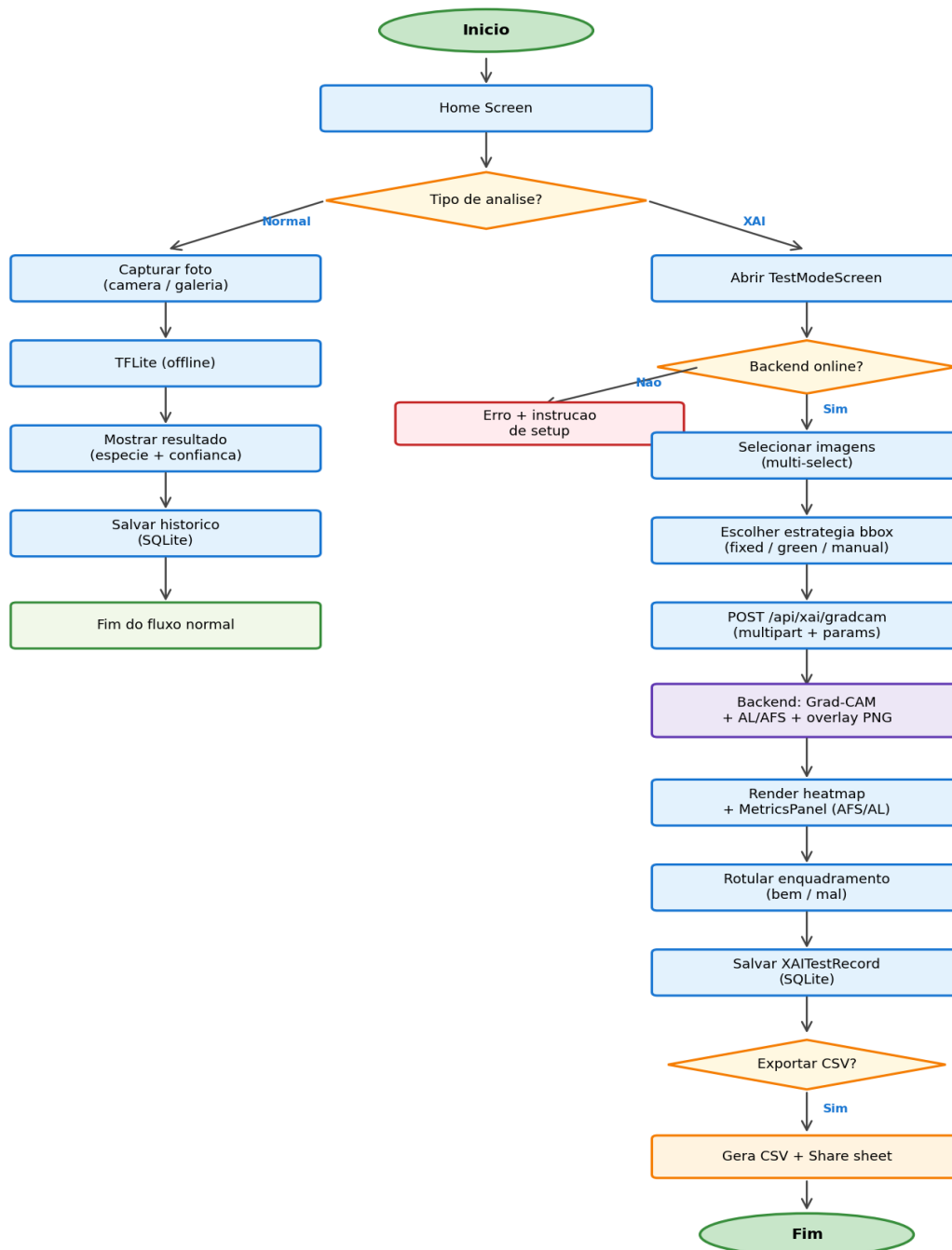


Figura 6. Fluxograma macro do AgroVision, cobrindo ambos os caminhos operacionais.

15. Discussão e trabalhos futuros

As métricas AL e AFS, por serem razões construídas sobre o heatmap Grad-CAM normalizado, herdam vantagens e limitações desse método. Entre as vantagens, destacam-se a facilidade de cálculo — duas operações vetoriais após o Grad-CAM já disponível — e a interpretabilidade direta em termos percentuais. Entre as limitações, observa-se que (i) a qualidade da métrica depende fortemente da qualidade da máscara M escolhida: bbox grosseiros tendem a reduzir a capacidade da métrica de discriminar vazamentos finos; (ii) o valor do *threshold* τ influencia a sensibilidade da métrica a ruído, demandando calibração quando se compara entre diferentes modelos ou *datasets*; (iii) AL/AFS capturam *vazamento espacial*, mas não explicam *qual* pista extraclasse o modelo está usando — essa análise requer inspeção qualitativa das regiões de alta ativação fora de M .

Entre os caminhos naturais de extensão deste trabalho incluem-se: substituir o bbox pela segmentação fina da planta obtida por um detector (ex.: YOLO ou MobileNet-SSD), o que estreita a fronteira da RoI e torna AL/AFS mais discriminativos; introduzir uma versão pixel-wise da métrica (baseada em IoU ponderado pela intensidade do heatmap); oferecer ao usuário final, dentro do app, o desenho interativo do bbox manual; e avaliar sistematicamente como AL/AFS evoluem ao longo de épocas de treinamento, fornecendo um sinal de convergência espacial complementar à *loss*.

16. Referências

- 1 Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). **Grad-CAM: Visual explanations from deep networks via gradient-based localization**. ICCV 2017, 618–626.
- 2 Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). **MobileNetV2: Inverted residuals and linear bottlenecks**. CVPR 2018, 4510–4520.
- 3 Zuiderveld, K. (1994). **Contrast Limited Adaptive Histogram Equalization**. In Graphics Gems IV (pp. 474–485). Academic Press.
- 4 Garcin, C., Joly, A., Bonnet, P., et al. (2021). **PI@ntNet-300K: a plant image dataset with high label ambiguity and a long-tailed distribution**. NeurIPS Datasets and Benchmarks Track.
- 5 Hughes, D. P., & Salathé, M. (2015). **An open access repository of images on plant health to enable the development of mobile disease diagnostics (PlantVillage)**. arXiv:1511.08060.
- 6 Abadi, M. et al. (2015). **TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems**. Disponível em: <https://www.tensorflow.org/>.
- 7 Ramírez, S. (2018–). **FastAPI: Modern, fast web framework for building APIs with Python**. Disponível em: <https://fastapi.tiangolo.com/>.
- 8 Meta. (2015–). **React Native Documentation**. Disponível em: <https://reactnative.dev/>.